

Freight Rate Prediction

- [Freight Rate Prediction](#)
 - [Results](#)
 - [Repository structure](#)
 - [Setup](#)
 - [Usage](#)
 - [Approach](#)
 - [Validation strategy](#)
 - [Known limitations / next steps](#)

Freight Rate Prediction

End-to-end pipeline that trains on historical freight loads and predicts `predicted_rate` for two required outputs: the 12,000-load validation set and the fixed-lane December trend chart, per the assessment instructions in `Freight_Rate_ML_Assessment.pdf`.

Results

5-fold, time-respecting backtest (see [Validation strategy](#)):

Metric	Value
RMSE	\$602
MAE	\$135
MAPE	6.0%
R ²	0.838

The provided `score.py` only validates output **format** and renders the December chart — it does not compute or expose an accuracy metric (that’s done externally by “Spotter” post-submission). The backtest above is reported for transparency since no target metric is specified in the assessment materials.

Repository structure

```
.
├── data/
│   ├── train_test.csv           # 48,000 labeled loads, Jan–Oct 2025
│   ├── validation.csv          # 12,000 loads to score, Nov–Dec 2025
│   ├── validation_predictions_template.csv # load_id + empty predicted_rate
│   └── december_chart_inputs.csv # 31 rows, one fixed lane, Dec 2025
├── freight_rate_pipeline.py     # training + inference pipeline
├── test_freight_rate_pipeline.py # pytest unit tests
├── score.py                     # provided scorer (validation + chart)
├── requirements.txt             # scorer dependencies
├── validation_predictions.csv   # output: filled template
├── december_chart_inputs.csv   # output: filled December file
└── README.md
```

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install "scikit-learn>=1.4" "pandas>=2.0,<3" "numpy>=1.26,<3"
```

Usage

Train and generate predictions:

```
python freight_rate_pipeline.py \
  --train-test data/train_test.csv \
  --validation data/validation.csv \
  --validation-template data/validation_predictions_template.csv \
  --december data/december_chart_inputs.csv \
  --output-dir .
```

Produces `validation_predictions.csv` and `december_chart_inputs.csv` in `--output-dir`. Use `--no-tune` to skip the ~25-candidate hyperparameter search and use sensible defaults (faster, slightly less accurate).

Validate outputs and generate the chart:

```
pip install -r requirements.txt
python score.py --predictions validation_predictions.csv \
  --december-predictions december_chart_inputs.csv
```

Run the test suite:

```
pip install pytest
pytest -v test_freight_rate_pipeline.py
```

Approach

Target. The model predicts `log1p(posted_rate / distance)` rather than raw dollars. `distance` alone explains ~91% of the variance in `posted_rate` (Pearson $r = 0.91$); modeling the ratio directly encodes that relationship structurally instead of asking the model to rediscover it, and keeps error behavior consistent across RMSE, MAE, and MAPE.

Features. `distance` (+ log), `weight` (median-imputed with a missingness flag), `equipment / pickup / delivery` as native categoricals, and calendar features — month, weekend flag, and Fourier (sine/cosine) encodings of day-of-year and day-of-week. The cyclical day-of-year encoding is the key choice for the December extrapolation: it lets the model recognize that December resembles January (both winter) using patterns learned from Jan–Oct, rather than requiring a literal chronological extrapolation trees can't do safely.

Features excluded. `market_index` and `quote_signal` are dropped. Their raw correlation with `posted_rate` is negligible (0.03–0.04, versus 0.91 for distance), and — more importantly — they don't exist in `december_chart_inputs.csv`. Using them would mean maintaining two divergent models or forecasting the forecast's own inputs. `evaluate_market_signal_ablation()` in the pipeline quantifies this trade-off empirically if you want to revisit it.

Model. `HistGradientBoostingRegressor` (scikit-learn) with native categorical support — avoids one-hot blow-up across 64–72 city categories and handles the mixed numeric/categorical feature set efficiently.

Validation strategy

Dates in `validation.csv` (Nov–Dec 2025) fall entirely after `train_test.csv` (Jan–Oct 2025) — this is a forecasting problem, not an i.i.d. holdout. All validation therefore uses `TimeSeriesSplit`, sorted chronologically, never a random k-fold, which would leak future months into training and mask exactly the risk that matters (extrapolation past October).

Hyperparameters are selected via `RandomizedSearchCV` over `TimeSeriesSplit`; reported metrics come from a separate, stricter fold-wise backtest that also refits feature-engineering statistics (e.g. median weight) inside each fold, to avoid any leakage between folds.

Known limitations / next steps

- Only 10 months of history are available, so there's no prior December to learn a genuine holiday-season effect from — the model's December pattern reflects weekly seasonality, not year-over-year holiday surge.
- `market_index / quote_signal` were excluded by design; if the target metric Spotter uses rewards squeezing out that residual signal, revisit via `evaluate_market_signal_ablation()`.
- Point estimates only. Adding `loss="quantile"` models would give prediction intervals, useful for communicating confidence per load.